DEPLOYMENT GUIDE

Intellia360 — AWS Deployment Guide

Step-by-step instructions for deploying Intellia360 to AWS for the first time, and for operating it afterwards.

Repository	`Intellia-SG/intellia360`
Applies to	the monorepo layout with `apps/frontend`, `apps/backend` and `packages/shared`
Frontend	Next.js static export, served by Amazon S3 + Amazon CloudFront
Backend API	Next.js Route Handlers on Amazon EC2 (Nginx + PM2), MongoDB Atlas
CI/CD	GitHub Actions with OpenID Connect (no stored AWS keys)
Estimated time	3–5 hours for the first deployment, most of it waiting for DNS and certificates
Version	1.0 — 26 September 2026

Contents

1. How to read this guide
   - 1.1 Conventions
   - 1.2 Order of work
2. What you are building
   - 2.1 The one rule you must not break: same parent domain
   - 2.2 Approximate monthly cost
3. Values worksheet
4. Phase 0 — Before you start
   - 4.1 Get the code onto `main`
   - 4.2 Accounts you need
   - 4.3 Tools
   - 4.4 Choose a region
5. Phase 1 — Harden the AWS account (15 minutes)
6. Phase 2 — MongoDB Atlas (20 minutes)
7. Phase 3 — EC2 server (40 minutes)
   - 7.1 IAM role for the server
   - 7.2 Security group
   - 7.3 Launch the instance
   - 7.4 Elastic IP
   - 7.5 Allow the server in Atlas
   - 7.6 Check that Session Manager works
8. Phase 4 — Set up the server (30 minutes)
   - 8.1 Packages, Node.js 22, PM2, swap
   - 8.2 App user and folders
   - 8.3 Read-only deploy key and repository mirror
   - 8.4 PM2 log rotation
9. Phase 5 — API environment file (15 minutes)
   - 9.1 Generate the secrets
   - 9.2 Create the file
10. Phase 6 — First API deploy (20 minutes)
    - 10.1 Run the deploy script
    - 10.2 Create the database indexes and the curriculum mirror (once)
    - 10.3 Create the platform admin (once, before going public)
11. Phase 7 — Nginx (10 minutes)
    - 11.1 Create the origin secret
    - 11.2 Install the site
    - 11.3 Test
12. Phase 8 — TLS certificate in ACM (10 minutes + waiting)
13. Phase 9 — CloudFront for the API (15 minutes + ~5 minutes deploying)
14. Phase 10 — Frontend hosting (30 minutes)

14.1 S3 bucket
14.2 Router function
14.3 Security headers policy
14.4 Frontend distribution

15. Phase 11 — DNS in Route 53 (15 minutes + propagation)
15.1 Hosted zone (skip if the domain was registered in Route 53)
15.2 Records

16. Phase 12 — GitHub Actions → AWS (30 minutes)
16.1 OIDC identity provider (once per AWS account)
16.2 Deploy role
16.3 GitHub settings
16.4 First pipeline runs

17. Phase 13 — Google sign-in and email
17.1 Google sign-in (optional)
17.2 Email sending domain (Resend)

18. Phase 14 — Monitoring and alerts (30 minutes)
18.1 CloudWatch agent on the server
18.2 Alert topic
18.3 Alarms
18.4 Uptime check

19. Phase 15 — Backups
19.1 Choose where backups run
19.2 Option B: nightly backup from the server to S3

20. Phase 16 — Go-live checks
20.1 From a terminal
20.2 In a browser (use a private window)
20.3 Final security checklist

21. Day-to-day operations
21.1 Deploying changes
21.2 Rolling back
21.3 Changing configuration
21.4 Database schema changes
21.5 Rotating secrets
21.6 Useful commands on the server

22. Troubleshooting

23. Known issue to fix before relying on the editor

24. Reference
24.1 Deployment files in the repository
24.2 Ports and names
24.3 GitHub configuration summary

1. How to read this guide

1.1 Conventions

Placeholders. Anything written as `<LIKE_THIS>` must be replaced, including the angle brackets. For example `<REGION>` becomes `ap-south-1`, not `<ap-south-1>`. Section 3 lists every placeholder; fill it in before you start.

Where to run each command. Every command block starts with a comment saying where it runs:

Label	Meaning
`# Run on: your computer`	a terminal on your own machine, inside the cloned repository
`# Run on: AWS CloudShell`	the browser terminal in the AWS console (icon `>_` in the top bar). It is already signed in as you and has the AWS CLI. Check the region shown in the console before running.
`# Run on: EC2 as root`	a Session Manager shell on the server, after running `sudo -i`
`# Run on: EC2 as intellia`	a Session Manager shell on the server, after running `sudo -iu intellia`

Quoting rules. These matter; most first-deployment failures come from quoting.

- **Shell values** that contain `$`, `!`, spaces or `&` must be in **single quotes**: `'p@ss$word!'`. Single quotes stop the shell from expanding anything inside them.
- `.env` **values** that contain spaces, `#`, `<`, `>` or quotes must be in **double quotes**: `RESEND_FROM="Intellia360 <noreply@yourdomain.com>"`. Do not put spaces around `=`.
- **JSON** always uses double quotes, never single quotes, and allows no trailing commas.
- **Special characters in the MongoDB password** must be URL-encoded inside `DATABASE_URL`, e.g. `@` → `%40`, `:` → `%3A`, `/` → `%2F`, `#` → `%23`, `?` → `%3F`. Using a password with only letters and digits avoids this entirely.
- **Paste commands exactly as written.** If a command spans several lines, the `\` at the end of each line is part of it.
- **Copy from the Markdown version when you can.** The PDF wraps long lines to fit the page, and a PDF viewer may copy that wrap as a line break. The same guide is in the repository as `docs/AWS-DEPLOYMENT-GUIDE.md`; copying from GitHub's rendered view keeps every command on its original lines.

Console instructions describe the setting names shown in the AWS console (for example **Viewer protocol policy**). AWS occasionally moves settings around; if a screen looks different, search the page for the setting name.

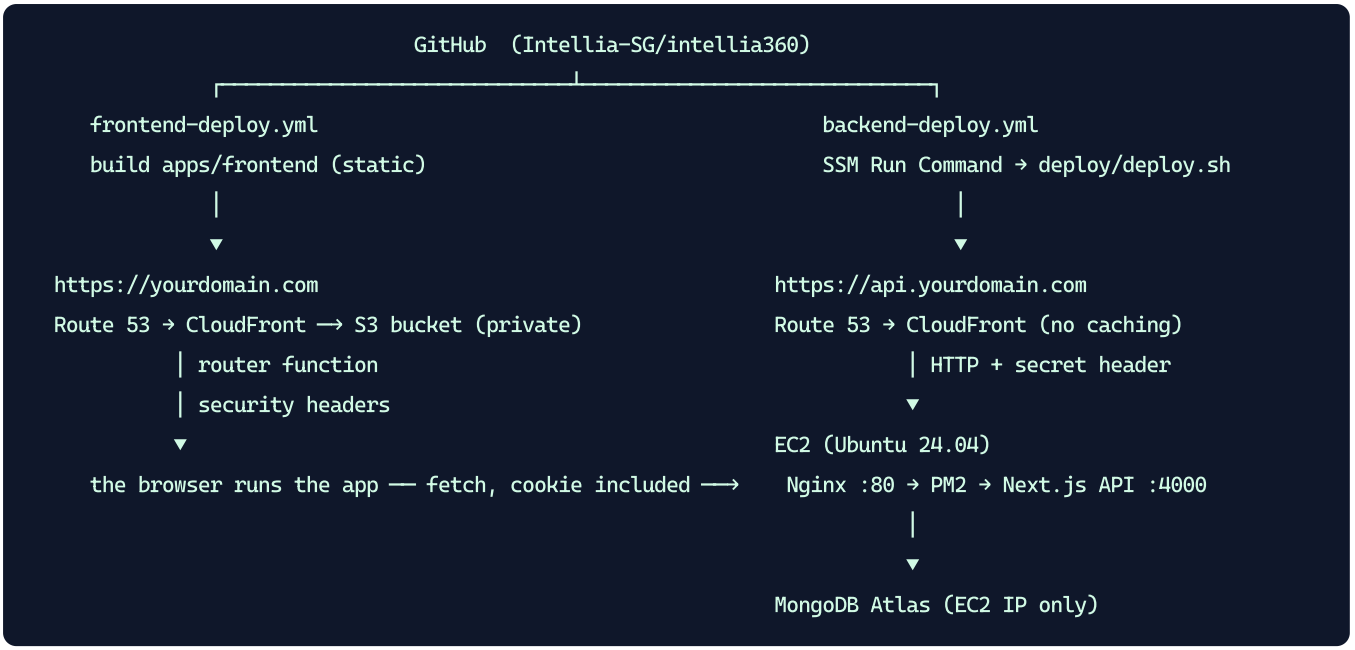
1.2 Order of work

Do the phases in order. Later phases need values created by earlier ones.

Phase	What you do	Section
0	Merge the code to `main`, prepare accounts and tools	4
1	Harden the AWS account	5
2	Create the MongoDB Atlas database	6

Phase	What you do	Section
3	Create the EC2 server (IAM role, security group, instance, Elastic IP)	7
4	Set up the server (Node.js, PM2, swap, deploy key, repository mirror)	8
5	Write the API environment file	9
6	First API deploy, database indexes, admin account	10
7	Nginx	11
8	TLS certificate (ACM)	12
9	CloudFront distribution for the API	13
10	S3 bucket, router function, headers policy, CloudFront for the frontend	14
11	DNS records (Route 53)	15
12	GitHub Actions → AWS (OIDC role, variables) and first pipeline runs	16
13	Google sign-in and email (Resend)	17
14	Monitoring and alerts	18
15	Backups	19
16	Go-live checks	20

2. What you are building



Component	Hosted on	Source in the repository
Web app (pages, UI, route guards)	S3 + CloudFront, as static files	`apps/frontend`
API (`/api/*`, auth, database, curriculum editor, AI tutor)	EC2, behind Nginx and PM2	`apps/backend`
Role rules and API types used by both	compiled into both	`packages/shared`
Server and CDN configuration	EC2, CloudFront	`deploy/`
Pipelines	GitHub Actions	`.github/workflows/`

2.1 The one rule you must not break: same parent domain

The login session is an `httpOnly` cookie named `intellia_session`, marked `Secure` and `SameSite=Lax`. The API sets it and the frontend sends it with every API call. Browsers only send it when the frontend and the API share the same registrable domain:

Frontend	API	Result
`https://yourdomain.com`	`https://api.yourdomain.com`	✅ works
`https://www.yourdomain.com`	`https://api.yourdomain.com`	✅ works
`https://d1234.cloudfront.net`	`https://api.yourdomain.com`	❌ sign-in fails
`https://yourdomain.com`	`https://intellia-api.com`	❌ sign-in fails

The `*.cloudfront.net` names are fine for quick connectivity tests, but not for signing in.

2.2 Approximate monthly cost

These are rough on-demand estimates for a small deployment in `ap-south-1` (Mumbai). Check them with the AWS Pricing Calculator before you commit.

Item	Approx. per month (USD)
EC2 `t3.small` (24×7)	~15–17
EBS 20 GB gp3	~2
Public IPv4 address (Elastic IP)	~3.6
CloudFront ×2 (small traffic, mostly within free tier)	0–5
S3 (a few hundred MB)	< 1
Route 53 hosted zone + queries	~0.5–1
ACM certificate	free
CloudWatch (agent metrics, logs, alarms)	~2–6
MongoDB Atlas	M0 free (development only); M10 ~57+ for production
Total (excluding Atlas)	**roughly 25–35**

3. Values worksheet

Fill this in as you go. Keep it somewhere private (a password manager). The secret values must **never** be committed to Git.

Placeholder	Meaning	Example	Your value
`<DOMAIN>`	the public domain	`yourdomain.com`	
`<API_DOMAIN>`	API host (a subdomain of `<DOMAIN>`)	`api.yourdomain.com`	
`<REGION>`	AWS region for EC2 and S3	`ap-south-1`	
`<ACCOUNT_ID>`	12-digit AWS account id	`123456789012`	
`<INSTANCE_ID>`	EC2 instance id	`i-0abc1234def567890`	
`<ELASTIC_IP>`	the server's fixed public IP	`13.234.10.20`	
`<EC2_PUBLIC_DNS>`	public DNS name of the Elastic IP	`ec2-13-234-10-20.ap-south-1.compute.amazonaws.com`	
`<ORIGIN_SECRET>`	random secret shared by CloudFront and Nginx	64 hex characters	🔒
`<BUCKET>`	S3 bucket for the frontend	`intellia360-frontend-prod`	
`<CERT_ARN>`	ACM certificate ARN (us-east-1)	`arn:aws:acm:us-east-1:…`	
`<API_DIST_ID>`	CloudFront distribution id for the API	`E2ABCDEFGHIJKL`	
`<API_DIST_DOMAIN>`	its `*.cloudfront.net` name	`d111111abcdef8.cloudfront.net`	
`<FRONTEND_DIST_ID>`	CloudFront distribution id for the frontend	`E3MNOPQRSTUVWX`	
`<FRONTEND_DIST_DOMAIN>`	its `*.cloudfront.net` name	`d222222abcdef8.cloudfront.net`	
`<DEPLOY_ROLE_ARN>`	IAM role GitHub Actions assumes	`arn:aws:iam::123456789012:role/intellia360-github-deploy`	
`<DATABASE_URL>`	Atlas connection string	`mongodb+srv://…`	🔒
`<JWT_SECRET>`	session signing secret	88 characters	🔒
`<ADMIN_EMAIL>` / `<ADMIN_PASSWORD>`	platform admin account	`admin@yourdomain.com`	🔒
`<GOOGLE_CLIENT_ID>`	Google OAuth client id (optional)	`1234-abc.apps.googleusercontent.com`	
`<RESEND_API_KEY>`	Resend API key	`re_…`	🔒
`<GROQ_API_KEY>`	Groq API key (AI tutor)	`gsk_…`	🔒
`<ALERT_EMAIL>`	who receives alarms	`ops@yourdomain.com`	

4. Phase 0 — Before you start

4.1 Get the code onto `main`

The deployment files live on the `chore/monorepo-restructure` branch. The workflows only run from `main`, and **Run workflow** only appears once a workflow file is on the default branch.

```
# Run on: your computer (inside the intellia360 repository)
git status
git commit -m "Split into frontend (S3/CloudFront) and backend (EC2) monorepo with shared package and AWS CI/CD"
git push -u origin chore/monorepo-restructure
```

Then, on GitHub, open a pull request from `chore/monorepo-restructure` into `main`. Its checks build both apps. **Merge it only after Phase 12 (section 16) is done**: merging triggers both deploy workflows, and they fail until AWS is ready. Alternatively, merge now and ignore the first failed runs.

4.2 Accounts you need

- **AWS account** with administrator access. Use an IAM Identity Center or IAM admin user, not the root user.
- **Domain name.** Either register it in Route 53, or keep your registrar and point its name servers at a Route 53 hosted zone (section 15.1).
- **MongoDB Atlas** account: https://cloud.mongodb.com
- **GitHub:** admin rights on `Intellia-SG/intellia360`.
- **Resend** account for email: https://resend.com
- **Optional:** Google Cloud project for Google sign-in, and a Groq API key for the AI tutor.

4.3 Tools

Everything in this guide can be done from the AWS console, AWS CloudShell and the GitHub website. Locally you only need `git`. The AWS CLI is optional, because CloudShell already has it.

4.4 Choose a region

Use one region for EC2 and S3, close to your users. For India that is `ap-south-1` (Mumbai). The TLS certificate is the exception: it **must** be in `us-east-1` (section 12). Write your region in the worksheet.

5. Phase 1 — Harden the AWS account (15 minutes)

1. **Root user:** enable MFA (IAM → Security recommendations), and stop using root for daily work.
2. **Admin access:** create an admin identity in IAM Identity Center (or an IAM user with `AdministratorAccess` and MFA) and use it from now on.
3. **Budget alert:** go to Billing and Cost Management → Budgets → Create budget → **Monthly cost budget**. Set it to e.g. `50` USD with an email alert, so surprise costs are noticed early.
4. **Account id:** note it in the worksheet (`<ACCOUNT_ID>`, top-right menu of the console).

6. Phase 2 — MongoDB Atlas (20 minutes)

1. **Create a cluster.** In Atlas, go to **Create** → cluster.
   - Production: **M10** or larger. It has backups and a dedicated resource allocation.
   - Trying things out: **M0** (free). It has no Atlas cloud backups.
   - Provider **AWS**, region the same as `<REGION>` (e.g. *Mumbai ap-south-1*).
2. **Create a database user.** Go to **Database Access** → **Add New Database User**.
   - Authentication: *Password*.
   - Username e.g. `intellia360_app`. Use a long password made of letters and digits only, so it needs no URL-encoding.
   - Built-in role: **Read and write to any database**. Better: use *Specific privileges* with `readWrite` on database `intellia360` only.
3. **Network Access.** Leave it for now. You will add the server's Elastic IP in section 7.5. **Never** add `0.0.0.0/0`.
4. **Get the connection string.** Go to **Connect** → **Drivers** and copy the string. Add the database name `intellia360` after the host:

```
mongodb+srv://intellia360_app:<DB_PASSWORD>@<CLUSTER_HOST>/intellia360?retryWrites=true&w=majority
```

This complete string is `<DATABASE_URL>`. The `/intellia360` part is the database name; the app's collections are created there.

7. Phase 3 — EC2 server (40 minutes)

All steps here are in region `<REGION>`. Check the region selector in the top right of the console.

7.1 IAM role for the server

This role lets you open shells through Session Manager (no SSH), lets GitHub deploys run through SSM, and lets the CloudWatch agent send logs.

Console:

1. Go to IAM → **Roles** → **Create role**.
2. Trusted entity: **AWS service**; use case **EC2** → **Next**.
3. Attach the policies `AmazonSSMManagedInstanceCore` and `CloudWatchAgentServerPolicy` → **Next**.
4. Role name `intellia360-ec2` → **Create role**.

Or with the CLI:

```
# Run on: AWS CloudShell
aws iam create-role --role-name intellia360-ec2 \
  --assume-role-policy-document '{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":
{"Service":"ec2.amazonaws.com"},"Action":"sts:AssumeRole"}]}'
aws iam attach-role-policy --role-name intellia360-ec2 \
  --policy-arn arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore
aws iam attach-role-policy --role-name intellia360-ec2 \
  --policy-arn arn:aws:iam::aws:policy/CloudWatchAgentServerPolicy
aws iam create-instance-profile --instance-profile-name intellia360-ec2
aws iam add-role-to-instance-profile --instance-profile-name intellia360-ec2 --role-name intellia360-ec2
```

7.2 Security group

The server accepts HTTP on port 80 **only from CloudFront**, and nothing else: no SSH, no direct web access. AWS publishes CloudFront's address ranges as the managed prefix list `com.amazonaws.global.cloudfront.origin-facing`.

```
# Run on: AWS CloudShell   (region = <REGION>)
PL=$(aws ec2 describe-managed-prefix-lists \
  --filters Name=prefix-list-name,Values=com.amazonaws.global.cloudfront.origin-facing \
  --query 'PrefixLists[0].PrefixListId' --output text)
echo "CloudFront prefix list: $PL"

SG=$(aws ec2 create-security-group --group-name intellia360-api \
  --description "Intellia360 API - HTTP from CloudFront only" \
  --query GroupId --output text)
echo "Security group: $SG"

aws ec2 authorize-security-group-ingress --group-id "$SG" \
  --ip-permissions "IpProtocol=tcp,FromPort=80,ToPort=80,PrefixListIds=
[{PrefixListId=$PL,Description=CloudFront}]"
```

> **Note.** The CloudFront prefix list counts as many rules toward the security group's rule quota. One rule like this fits within the default quota; don't add other large prefix lists to the same group.

Console alternative: EC2 → **Security Groups** → **Create security group**.

- Name `intellia360-api`, VPC *default*.
- Inbound rule: Type **HTTP**, Source **Custom** → start typing `com.amazonaws.global.cloudfront.origin-facing` and select the prefix list.
- Leave outbound as default (all traffic).
- No SSH rule.

7.3 Launch the instance

EC2 → **Instances** → **Launch instances**:

Setting	Value
Name	`intellia360-api`
AMI	**Ubuntu Server 24.04 LTS (HVM), SSD Volume Type**, 64-bit (x86)
Instance type	`t3.small` (2 GiB), or `t3.medium` (4 GiB) for faster builds
Key pair	**Proceed without a key pair** (you will use Session Manager)
Network	default VPC, a public subnet, **Auto-assign public IP: Enable**
Firewall	**Select existing security group** → `intellia360-api`
Storage	`20` GiB, **gp3**
Advanced → IAM instance profile	`intellia360-ec2`
Advanced → Metadata version	**V2 only (token required)**

Click **Launch instance** and copy the **instance id** into the worksheet (`<INSTANCE_ID>`).

7.4 Elastic IP

The public IP must never change: Atlas allowlists it, and CloudFront connects to its DNS name.

1. EC2 → **Elastic IPs** → **Allocate Elastic IP address** → **Allocate**.
2. Select it → **Actions** → **Associate Elastic IP address** → Instance `intellia360-api` → **Associate**.
3. Open the instance and copy **Public IPv4 address** (`<ELASTIC_IP>`) and **Public IPv4 DNS** (`<EC2_PUBLIC_DNS>`, e.g. `ec2-13-234-10-20.ap-south-1.compute.amazonaws.com`).

7.5 Allow the server in Atlas

Atlas → **Network Access** → **Add IP Address** → enter `<ELASTIC_IP>` (it becomes `<ELASTIC_IP>/32`), comment `intellia360 EC2` → **Confirm**.

7.6 Check that Session Manager works

EC2 → select the instance → **Connect** → **Session Manager** tab → **Connect**. A terminal opens in the browser. If the tab says the instance is not connected, wait 5 minutes and check that:

- the instance has the `intellia360-ec2` role (7.1);
- it has a public IP, so the agent can reach AWS;
- the Ubuntu AMI was used (it includes the SSM agent).

Every "Run on: EC2" block in this guide is typed into this Session Manager terminal.

8. Phase 4 — Set up the server (30 minutes)

8.1 Packages, Node.js 22, PM2, swap

```
# Run on: EC2 as root   (first type: sudo -i)
apt-get update && apt-get -y upgrade
apt-get install -y nginx git curl jq

# Node.js 22 LTS from NodeSource
curl -fsSL https://deb.nodesource.com/setup_22.x | bash -
apt-get install -y nodejs

# PM2 process manager
npm install -g pm2

# 2 GB swap: `next build` needs more memory than a t3.small has
fallocate -l 2G /swapfile && chmod 600 /swapfile && mkswap /swapfile && swapon /swapfile
echo '/swapfile none swap sw 0 0' >> /etc/fstab

# Check versions
node -v     # expect v22.x
npm -v      # expect 10.x or newer
pm2 -v
free -h     # "Swap:" should show 2.0Gi
```

8.2 App user and folders

The API runs as an unprivileged user `intellia`. All deploy files live in `/srv/intellia360`.

```
# Run on: EC2 as root
useradd --create-home --shell /bin/bash intellia
mkdir -p /srv/intellia360/releases /srv/intellia360/shared/backups
chown -R intellia:intellia /srv/intellia360

# Start PM2 on boot, as the intellia user
env PATH=$PATH:/usr/bin pm2 startup systemd -u intellia --hp /home/intellia
```

The resulting layout (`deploy/deploy.sh` depends on it):

```
/srv/intellia360/
├── repo.git/          bare mirror of the GitHub repository (created in 8.3)
├── shared/.env        API environment and secrets (section 9)
├── shared/backups/    local database backup archives
├── releases/<stamp>/  one folder per deploy, created by deploy.sh
└── current ->         symlink to the live release
```

8.3 Read-only deploy key and repository mirror

The server pulls code from GitHub with its own SSH key, which can **only read** this one repository.

```
# Run on: EC2 as intellia   (first type: sudo -iu intellia)
ssh-keygen -t ed25519 -f ~/.ssh/intellia360_deploy -N "" -C "intellia360-ec2"
cat ~/.ssh/intellia360_deploy.pub
```

Copy the printed line (it starts with `ssh-ed25519`). On GitHub, go to **Intellia-SG/intellia360** → **Settings** → **Deploy keys** → **Add deploy key**:

- Title `intellia360-ec2`
- Key: paste the line
- **Allow write access: leave unticked**

Click **Add key**. Then:

```
# Run on: EC2 as intellia
cat >> ~/.ssh/config <<'EOF'
Host github.com
  IdentityFile ~/.ssh/intellia360_deploy
  IdentitiesOnly yes
EOF
chmod 600 ~/.ssh/config
ssh-keyscan github.com >> ~/.ssh/known_hosts

ssh -T git@github.com
# expect: "Hi Intellia-SG/intellia360! You've successfully authenticated, but GitHub does not provide shell
access."

git clone --mirror git@github.com:Intellia-SG/intellia360.git /srv/intellia360/repo.git
git -C /srv/intellia360/repo.git branch --list
# expect: main (and chore/monorepo-restructure until it is deleted)
```

8.4 PM2 log rotation

```
# Run on: EC2 as intellia
pm2 install pm2-logrotate
pm2 set pm2-logrotate:max_size 20M
pm2 set pm2-logrotate:retain 14
```

9. Phase 5 — API environment file (15 minutes)

The API reads its settings from `/srv/intellia360/shared/.env`. Every deploy links this file into the new release as `apps/backend/.env`. It is the **only place** API secrets exist.

9.1 Generate the secrets

```
# Run on: EC2 as intellia
openssl rand -base64 64 | tr -d '\n'; echo     # → <JWT_SECRET>  (88 characters)
```

`<ADMIN_PASSWORD>` must have at least 8 characters, including a letter, a digit and a special character (the app's password rule). Use 16+ characters.

9.2 Create the file

Start from the template in the repository, then edit it:

```
# Run on: EC2 as intellia
git -C /srv/intellia360/repo.git show main:apps/backend/.env.example > /srv/intellia360/shared/.env
chmod 600 /srv/intellia360/shared/.env
nano /srv/intellia360/shared/.env
```

Make the file contain these values (replace every placeholder; keep the double quotes shown):

```
# ── Database ──────────────────────────────────────────
DATABASE_URL="<DATABASE_URL>"

# ── Sessions ──────────────────────────────────────────
AUTH_JWT_SECRET="<JWT_SECRET>"
AUTH_JWT_EXPIRES_IN=7d

# ── Platform admin (used once, section 10.3) ──────────
ADMIN_EMAIL="<ADMIN_EMAIL>"
ADMIN_NAME="Platform Admin"
ADMIN_PASSWORD="<ADMIN_PASSWORD>"

# ── Frontend URL (links in emails) and CORS ───────────
NEXT_PUBLIC_APP_URL="https://<DOMAIN>"
FRONTEND_URL="https://<DOMAIN>"
CORS_ORIGIN="https://<DOMAIN>,https://www.<DOMAIN>"

# ── Email (Resend) ────────────────────────────────────
RESEND_API_KEY="<RESEND_API_KEY>"
RESEND_FROM="Intellia360 <onboarding@resend.dev>"
AUTH_REQUIRE_EMAIL_VERIFICATION=false

# ── Google sign-in (optional) ─────────────────────────
GOOGLE_CLIENT_ID="<GOOGLE_CLIENT_ID>"
```

```
# ── AI tutor (Groq) ──────────────────────────────────
GROQ_API_KEY="<GROQ_API_KEY>"

# ── Backups ──────────────────────────────────────────
BACKUP_DIR=/srv/intellia360/shared/backups
BACKUP_KEEP=14
```

Save in nano with `Ctrl+O`, `Enter`, `Ctrl+X`.

Rules for this file:

Variable	Notes
`CORS_ORIGIN`	exact browser origins: scheme + host, **no path and no trailing slash**, comma-separated with no spaces. Any other site is refused.
`NEXT_PUBLIC_APP_URL`, `FRONTEND_URL`	the **frontend** address, not the API
`RESEND_FROM`	keep `onboarding@resend.dev` until your domain is verified in Resend (section 17.2)
`AUTH_REQUIRE_EMAIL_VERIFICATION`	switch to `true` only after the sending domain is verified
`GOOGLE_CLIENT_ID`	leave the line out if you don't use Google sign-in
`NEXT_PUBLIC_*`	compiled in at build time. After changing one, **redeploy** the API; a restart is not enough.

Check that nothing is left unreplaced:

```
# Run on: EC2 as intellia
grep -n '<' /srv/intellia360/shared/.env | grep -v 'RESEND_FROM'   # should print nothing
```

10. Phase 6 — First API deploy (20 minutes)

10.1 Run the deploy script

The script builds `apps/backend` in a new release folder, switches `current` to it, starts PM2 and checks `/api/health`. The first run takes 5–10 minutes.

```
# Run on: EC2 as intellia
cd /srv/intellia360
git -C repo.git show main:deploy/deploy.sh | bash -s -- main
```

It should end with:

```
[deploy] deployed <commit sha>
```

> If the branch is not merged yet, deploy the branch instead: replace both `main` in the command with `chore/monorepo-restructure`.

Check it:

```
# Run on: EC2 as intellia
pm2 status                                  # intellia360-api  online
curl -s http://127.0.0.1:4000/api/health    # {"status":"ok"}
```

If the health check reports `{"status":"unavailable"}`, the API cannot reach MongoDB. Check `DATABASE_URL` and the Atlas IP access list (7.5).

10.2 Create the database indexes and the curriculum mirror (once)

```
# Run on: EC2 as intellia
cd /srv/intellia360/current
npm run db:indexes
npm run db:restore-curriculum -- --seed-mirror
```

- `db:indexes` runs `prisma db push`, which creates the collections and indexes.
- `--seed-mirror` copies the curriculum files that ship with the code into the database, so editor history can be recovered later.

10.3 Create the platform admin (once, before going public)

`POST /api/admin/seed-admin` creates the first admin from `ADMIN_EMAIL`, `ADMIN_NAME` and `ADMIN_PASSWORD`. It needs no login until an admin exists, so run it now, while the API is reachable only from the server itself:

```
# Run on: EC2 as intellia
curl -s -X POST http://127.0.0.1:4000/api/admin/seed-admin; echo
```

Run it a second time to confirm; it should now report that an admin already exists (HTTP 410).

> ⚠️ **Do not run `npm run seed` in production.** It also creates demo teacher and student accounts that all share the password `Password123!`.

Afterwards you may remove `ADMIN_PASSWORD` from `shared/.env`. It is only used for this bootstrap.

11. Phase 7 — Nginx (10 minutes)

Nginx accepts requests on port 80 and passes them to the API on `127.0.0.1:4000`. It **rejects any request without the secret header** that CloudFront adds, and it records the real visitor IP for rate limiting.

11.1 Create the origin secret

```
# Run on: EC2 as root
ORIGIN_SECRET=$(openssl rand -hex 32)
echo "$ORIGIN_SECRET"          # copy into the worksheet as <ORIGIN_SECRET>

cat > /etc/nginx/snippets/intellia360-origin-verify.conf <<EOF
if (\$http_x_origin_verify != "$ORIGIN_SECRET") { return 403; }
EOF
chmod 640 /etc/nginx/snippets/intellia360-origin-verify.conf
cat /etc/nginx/snippets/intellia360-origin-verify.conf
# expect: if ($http_x_origin_verify != "<64 hex characters>") { return 403; }
```

> The `\$` before `http_x_origin_verify` is deliberate: it keeps the shell from treating it as a variable, so Nginx receives `$http_x_origin_verify`.

11.2 Install the site

```
# Run on: EC2 as root
cp /srv/intellia360/current/deploy/nginx/intellia360.conf /etc/nginx/sites-available/intellia360
ln -sf /etc/nginx/sites-available/intellia360 /etc/nginx/sites-enabled/intellia360
rm -f /etc/nginx/sites-enabled/default
nginx -t && systemctl reload nginx
```

`nginx -t` must say `syntax is ok` and `test is successful`.

11.3 Test

```
# Run on: EC2 as root
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1/api/health
# expect 403  (no secret header)

curl -s -H "X-Origin-Verify: $ORIGIN_SECRET" http://127.0.0.1/api/health; echo
# expect {"status":"ok"}
```

If you opened a new shell since 11.1, `$ORIGIN_SECRET` is empty. Paste the value in single quotes instead: `-H 'X-Origin-Verify: <ORIGIN_SECRET>'`.

> If `deploy/nginx/intellia360.conf` changes in a later release, repeat 11.2. Deploys never modify `/etc/nginx`.

12. Phase 8 — TLS certificate in ACM (10 minutes + waiting)

CloudFront only uses certificates from **US East (N. Virginia)** `us-east-1`, whatever region the rest of the system is in. One certificate covers all three hosts.

1. Switch the console region to **US East (N. Virginia)**.
2. Go to Certificate Manager → **Request** → **Request a public certificate** → **Next**.
3. Fully qualified domain names:
   - `<DOMAIN>`
   - `www.<DOMAIN>`
   - `<API_DOMAIN>`
4. Validation method **DNS validation**; key algorithm **RSA 2048** → **Request**.
5. Open the certificate → **Create records in Route 53** → select all → **Create records**. (If your DNS is not in Route 53 yet, do section 15.1 first.)
6. Wait until the status is **Issued**, usually 5–30 minutes. Copy the ARN into the worksheet (`<CERT_ARN>`).

CLI alternative:

```
# Run on: AWS CloudShell
aws acm request-certificate --region us-east-1 \
  --domain-name '<DOMAIN>' \
  --subject-alternative-names 'www.<DOMAIN>' '<API_DOMAIN>' \
  --validation-method DNS
```

13. Phase 9 — CloudFront for the API (15 minutes + ~5 minutes deploying)

CloudFront → **Distributions** → **Create distribution**. Use these settings; anything not listed stays at its default.

Origin

Setting	Value
Origin domain	`<EC2_PUBLIC_DNS>`. Type it in; CloudFront does not accept IP addresses.
Protocol	**HTTP only**, HTTP port `80`
Name	`intellia360-ec2`
Add custom header	Header name `X-Origin-Verify`, value `<ORIGIN_SECRET>`
Response timeout	`60` seconds (AI tutor answers can be slow)

Default cache behavior

Setting	Value
Viewer protocol policy	**Redirect HTTP to HTTPS**
Allowed HTTP methods	**GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE**
Cache policy	**CachingDisabled**
Origin request policy	**AllViewerAndCloudFrontHeaders-2022-06**
Response headers policy	none

Web Application Firewall: Do not enable security protections. It is optional and costs extra; you can add it later.

Settings

Setting	Value
Price class	the class that includes your users, e.g. *Use North America, Europe, Asia, Middle East, and Africa*
Alternate domain name (CNAME)	`<API_DOMAIN>`
Custom SSL certificate	the certificate from section 12
Security policy	**TLSv1.2_2021**
Supported HTTP versions	HTTP/2 (and HTTP/3)
IPv6	On

Click **Create distribution**. Copy the distribution **ID** (`<API_DIST_ID>`) and **domain name** (`<API_DIST_DOMAIN>`).

Why these settings:

- **CachingDisabled:** every API response is personal and must never be cached.
- **AllViewerAndCloudFrontHeaders:**

    - forwards `Origin`, which the API's CORS check needs;
    - forwards the session cookie and `Host`;
    - forwards `CloudFront-Viewer-Address`, which Nginx turns into the real client IP.
- **Custom header:** without it Nginx answers 403 (section 11).

Test it once the status is *Deployed*:

```
# Run on: your computer or AWS CloudShell
curl -s https://<API_DIST_DOMAIN>/api/health; echo
# expect {"status":"ok"}
```

14. Phase 10 — Frontend hosting (30 minutes)

14.1 S3 bucket

S3 (region `<REGION>`) → **Create bucket**:

Setting	Value
Bucket name	`<BUCKET>` (globally unique, e.g. `intellia360-frontend-prod`)
Object Ownership	**ACLs disabled**
Block all public access	**On** (all four boxes ticked)
Bucket Versioning	**Enable** (lets you restore a previous upload)
Default encryption	SSE-S3

The bucket stays private. Only CloudFront will read it (14.4).

14.2 Router function

This function maps clean URLs onto the exported files: `/login` → `/login.html`, and `/student/lesson/<any id>` → the lesson shell.

1. CloudFront → **Functions** → **Create function**.
2. Name `intellia360-frontend-router`, runtime **cloudfront-js-2.0** → **Create function**.
3. In the **Build** tab, replace the example code with the **entire contents of** `deploy/cloudfront/frontend-router.js` from the repository → **Save changes**.
4. **Test** tab: event type *Viewer request*, URL path `/login` → **Test function**. The output request URI should be `/login.html`.
5. **Publish** tab → **Publish function**.

Later frontend deploys republish this function automatically (variable `FRONTEND_ROUTER_FUNCTION_NAME`, section 16.3).

14.3 Security headers policy

These are the browser security headers the app used to set itself.

1. Open `deploy/cloudfront/frontend-security-headers.json` from the repository.
2. Replace `https://api.yourdomain.com` with `https://<API_DOMAIN>`. It appears once, inside `connect-src`.
3. In CloudShell, upload the edited file: **Actions** → **Upload file**.
4. Run:

```
# Run on: AWS CloudShell   (after uploading the edited JSON file)
aws cloudfront create-response-headers-policy \
  --response-headers-policy-config file://frontend-security-headers.json
```

Console alternative: CloudFront → **Policies** → **Response headers** → **Create response headers policy**. Name `intellia360-frontend-security-headers`, then tick and fill:

Header	Value	Origin override
Strict-Transport-Security	max age `31536000`, *includeSubDomains* ✓, *preload* ✓	✓
X-Content-Type-Options	nosniff	✓
X-Frame-Options	`SAMEORIGIN`	✓
Referrer-Policy	`strict-origin-when-cross-origin`	✓
Content-Security-Policy	the full `ContentSecurityPolicy` string from the JSON file, with your API domain	✓
Custom header `Permissions-Policy`	`camera=(), microphone=(), geolocation=(), interest-cohort=()`	✓

14.4 Frontend distribution

CloudFront → **Distributions** → **Create distribution**:

Origin

Setting	Value
Origin domain	pick `<BUCKET>.s3.<REGION>.amazonaws.com` from the list. Use the bucket, **not** a "website endpoint".
Origin access	**Origin access control settings (recommended)** → **Create new OAC** (defaults: sign requests)

Default cache behavior

Setting	Value
Viewer protocol policy	**Redirect HTTP to HTTPS**
Allowed HTTP methods	**GET, HEAD**
Cache policy	**CachingOptimized**
Origin request policy	none
Response headers policy	**intellia360-frontend-security-headers**
Function associations → Viewer request	**CloudFront Functions** → **intellia360-frontend-router**
Compress objects automatically	Yes

Settings

Setting	Value
Alternate domain names	`<DOMAIN>` and `www.<DOMAIN>`
Custom SSL certificate	the certificate from section 12; security policy **TLSv1.2_2021**
Default root object	`index.html`
Price class	same as the API distribution

Click **Create distribution**.

1. **Bucket policy:** CloudFront shows a banner **"The S3 bucket policy needs to be updated"** → **Copy policy** → open the bucket → **Permissions** → **Bucket policy** → **Edit** → paste → **Save**. The policy looks like this:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontServicePrincipalReadOnly",
      "Effect": "Allow",
      "Principal": { "Service": "cloudfront.amazonaws.com" },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::<BUCKET>/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::<ACCOUNT_ID>:distribution/<FRONTEND_DIST_ID>"
        }
      }
    }
  ]
}
```

2. **Error pages:** open the distribution → **Error pages** → **Create custom error response**, twice:

HTTP error code	Customize error response	Response page path	HTTP response code
`403`	Yes	`/404.html`	`404`
`404`	Yes	`/404.html`	`404`

A private bucket answers **403** for files that don't exist, which is why 403 is mapped too.

Copy the **ID** (`<FRONTEND_DIST_ID>`) and **domain name** (`<FRONTEND_DIST_DOMAIN>`).

The bucket is still empty; the first frontend deploy (section 16.4) fills it.

15. Phase 11 — DNS in Route 53 (15 minutes + propagation)

15.1 Hosted zone (skip if the domain was registered in Route 53)

1. Route 53 → **Hosted zones** → **Create hosted zone** → domain `<DOMAIN>`, type *Public*.
2. Copy the four **NS** name servers into your registrar's name server settings.
3. Wait for propagation (minutes to 48 hours). Check with:

```
# Run on: your computer
nslookup -type=NS <DOMAIN>
```

15.2 Records

Route 53 → the `<DOMAIN>` hosted zone → **Create record**. Create six records, all with **Alias = Yes** and target *Alias to CloudFront distribution*:

Record name	Type	Alias target
(empty — the apex)	A	frontend distribution (`<FRONTEND_DIST_DOMAIN>`)
(empty — the apex)	AAAA	frontend distribution
`www`	A	frontend distribution
`www`	AAAA	frontend distribution
`api`	A	API distribution (`<API_DIST_DOMAIN>`)
`api`	AAAA	API distribution

CLI alternative (the CloudFront alias hosted zone id is always `Z2FDTNDATAQYW2`):

```
# Run on: AWS CloudShell
ZONE_ID=$(aws route53 list-hosted-zones-by-name --dns-name '<DOMAIN>' \
  --query 'HostedZones[0].Id' --output text)

cat > dns.json <<'EOF'
{
  "Changes": [
    { "Action": "UPSERT", "ResourceRecordSet": { "Name": "<DOMAIN>", "Type": "A",
      "AliasTarget": { "HostedZoneId": "Z2FDTNDATAQYW2", "DNSName": "<FRONTEND_DIST_DOMAIN>",
"EvaluateTargetHealth": false } } },
    { "Action": "UPSERT", "ResourceRecordSet": { "Name": "<DOMAIN>", "Type": "AAAA",
      "AliasTarget": { "HostedZoneId": "Z2FDTNDATAQYW2", "DNSName": "<FRONTEND_DIST_DOMAIN>",
"EvaluateTargetHealth": false } } },
    { "Action": "UPSERT", "ResourceRecordSet": { "Name": "www.<DOMAIN>", "Type": "A",
      "AliasTarget": { "HostedZoneId": "Z2FDTNDATAQYW2", "DNSName": "<FRONTEND_DIST_DOMAIN>",
"EvaluateTargetHealth": false } } },
    { "Action": "UPSERT", "ResourceRecordSet": { "Name": "www.<DOMAIN>", "Type": "AAAA",
      "AliasTarget": { "HostedZoneId": "Z2FDTNDATAQYW2", "DNSName": "<FRONTEND_DIST_DOMAIN>",
"EvaluateTargetHealth": false } } },
```

```
    { "Action": "UPSERT", "ResourceRecordSet": { "Name": "<API_DOMAIN>", "Type": "A",
      "AliasTarget": { "HostedZoneId": "Z2FDTNDATAQYW2", "DNSName": "<API_DIST_DOMAIN>",
"EvaluateTargetHealth": false } } },
    { "Action": "UPSERT", "ResourceRecordSet": { "Name": "<API_DOMAIN>", "Type": "AAAA",
      "AliasTarget": { "HostedZoneId": "Z2FDTNDATAQYW2", "DNSName": "<API_DIST_DOMAIN>",
"EvaluateTargetHealth": false } } }
  ]
}
EOF
aws route53 change-resource-record-sets --hosted-zone-id "$ZONE_ID" --change-batch file://dns.json
```

Test (it may take a few minutes):

```
# Run on: your computer
curl -s https://<API_DOMAIN>/api/health; echo      # {"status":"ok"}
```

16. Phase 12 — GitHub Actions → AWS (30 minutes)

GitHub Actions deploys **without any stored AWS access keys**. Each run proves its identity to AWS with a short-lived OpenID Connect token and receives temporary credentials for one IAM role. That role is limited to what the two deploys need.

16.1 OIDC identity provider (once per AWS account)

IAM → **Identity providers** → **Add provider**:

Setting	Value
Provider type	**OpenID Connect**
Provider URL	`https://token.actions.githubusercontent.com`
Audience	`sts.amazonaws.com`

Click **Add provider**. If one with this URL already exists, reuse it.

16.2 Deploy role

1. IAM → **Roles** → **Create role** → **Custom trust policy**. Paste the following, replacing `<ACCOUNT_ID>`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::<ACCOUNT_ID>:oidc-provider/token.actions.githubusercontent.com"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
          "token.actions.githubusercontent.com:sub": "repo:Intellia-SG/intellia360:environment:production"
        }
      }
    }
  ]
}
```

The `sub` condition means only jobs of this repository that run in its `production` environment can use the role.

2. **Next** → skip attaching managed policies → **Next** → name `intellia360-github-deploy` → **Create role**.
3. Open the role → **Add permissions** → **Create inline policy** → **JSON**. Paste the following, replacing `<REGION>`, `<ACCOUNT_ID>`, `<INSTANCE_ID>`, `<BUCKET>` and `<FRONTEND_DIST_ID>`:

```
{
  "Version": "2012-10-17",
```

```
  "Statement": [
    {
      "Sid": "BackendDeployViaSsm",
      "Effect": "Allow",
      "Action": "ssm:SendCommand",
      "Resource": [
        "arn:aws:ec2:<REGION>:<ACCOUNT_ID>:instance/<INSTANCE_ID>",
        "arn:aws:ssm:<REGION>::document/AWS-RunShellScript"
      ]
    },
    {
      "Sid": "BackendDeployStatus",
      "Effect": "Allow",
      "Action": "ssm:GetCommandInvocation",
      "Resource": "*"
    },
    {
      "Sid": "FrontendUpload",
      "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
      "Resource": ["arn:aws:s3:::<BUCKET>", "arn:aws:s3:::<BUCKET>/*"]
    },
    {
      "Sid": "FrontendInvalidate",
      "Effect": "Allow",
      "Action": "cloudfront:CreateInvalidation",
      "Resource": "arn:aws:cloudfront::<ACCOUNT_ID>:distribution/<FRONTEND_DIST_ID>"
    },
    {
      "Sid": "FrontendRouterFunction",
      "Effect": "Allow",
      "Action": ["cloudfront:DescribeFunction", "cloudfront:UpdateFunction", "cloudfront:PublishFunction"],
      "Resource": "arn:aws:cloudfront::<ACCOUNT_ID>:function/intellia360-frontend-router"
    }
  ]
}
```

Name it `intellia360-deploy` → **Create policy**. 4. Copy the role **ARN** (`<DEPLOY_ROLE_ARN>`).

16.3 GitHub settings

On GitHub, go to **Intellia-SG/intellia360** → **Settings**.

1. Environment. Environments → **New environment** → name `production` (exactly, lowercase) → **Configure environment**.

- Optional but recommended: **Required reviewers**. Every deploy then waits for someone to approve it.
- **Deployment branches and tags** → *Selected branches and tags* → add `main`.

2. Secret. Secrets and variables → **Actions** → **Secrets** tab → **New repository secret**:

Name	Value
`AWS_DEPLOY_ROLE_ARN`	`<DEPLOY_ROLE_ARN>`

3. Variables. Same page → **Variables** tab → **New repository variable** for each row.

> These must be **repository** variables, not environment variables: the frontend build job reads them and does not run in the `production` environment.

Name	Value
`AWS_REGION`	`<REGION>`
`EC2_INSTANCE_ID`	`<INSTANCE_ID>`
`FRONTEND_S3_BUCKET`	`<BUCKET>`
`FRONTEND_CLOUDFRONT_DISTRIBUTION_ID`	`<FRONTEND_DIST_ID>`
`FRONTEND_ROUTER_FUNCTION_NAME`	`intellia360-frontend-router`
`NEXT_PUBLIC_API_URL`	`https://<API_DOMAIN>` (no trailing slash)
`NEXT_PUBLIC_GOOGLE_CLIENT_ID`	`<GOOGLE_CLIENT_ID>`, or leave it out

`NEXT_PUBLIC_*` values are compiled into the JavaScript every visitor downloads, so they are **not secret**. Never put a secret in a `NEXT_PUBLIC_*` variable.

16.4 First pipeline runs

Merge the pull request from section 4.1 into `main`. This starts both workflows. Or, if it is already merged, go to **Actions** → pick the workflow → **Run workflow** → branch `main`.

Workflow	What it does	Success looks like
Frontend — build & deploy to S3 + CloudFront	builds the static site, checks routing, uploads to S3, publishes the router function, invalidates the cache	both jobs green; `https://<DOMAIN>` shows the home page
Backend — build & deploy to EC2	type-checks and builds, then runs `deploy/deploy.sh` on the server through SSM	both jobs green; the log ends with `Deploy status: Success`

If a run waits with "*Waiting for review*", approve it in the run page (you enabled required reviewers).

17. Phase 13 — Google sign-in and email

17.1 Google sign-in (optional)

In the Google Cloud console → **APIs & Services** → **Credentials** → your **OAuth 2.0 Client ID** (type *Web application*) → **Authorised JavaScript origins** → add:

```
https://<DOMAIN>
https://www.<DOMAIN>
```

No redirect URIs are needed; the app uses the ID-token flow. The same client id must be set in:

- `GOOGLE_CLIENT_ID` in `shared/.env`. Then run `pm2 restart intellia360-api` as `intellia`.
- **GitHub variable** `NEXT_PUBLIC_GOOGLE_CLIENT_ID`. Then re-run the frontend workflow.

17.2 Email sending domain (Resend)

1. Resend → **Domains** → **Add domain** → `<DOMAIN>`.
2. Resend shows DNS records (TXT for SPF, CNAME/TXT for DKIM, optionally MX). Create **exactly** those records in the Route 53 hosted zone. Copy names and values character for character.
3. Click **Verify** in Resend and wait for *Verified*.
4. Update `shared/.env`:

```
RESEND_FROM="Intellia360 <noreply@<DOMAIN>>"
AUTH_REQUIRE_EMAIL_VERIFICATION=true
```

5. Restart the API:

```
# Run on: EC2 as intellia
pm2 restart intellia360-api
```

Test by registering an account and checking that the verification email arrives and its link opens `https://<DOMAIN>/verify-email?...`.

18. Phase 14 — Monitoring and alerts (30 minutes)

18.1 CloudWatch agent on the server

It ships PM2 and Nginx logs, plus memory, disk and swap usage, to CloudWatch.

```
# Run on: EC2 as root
curl -fsSLo /tmp/cwagent.deb https://amazoncloudwatch-agent.s3.amazonaws.com/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb
dpkg -i /tmp/cwagent.deb
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a fetch-config -m ec2 -s \
  -c file:/srv/intellia360/current/deploy/cloudwatch-agent.json
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a status   # "status": "running"
```

What you get in CloudWatch:

Where	Contents
Log group `/intellia360/app`	API output (PM2 stdout / stderr)
Log group `/intellia360/nginx`	Nginx access and error logs
Log group `/intellia360/deploy`	output of every GitHub deploy
Metrics namespace `Intellia360`	`mem_used_percent`, `swap_used_percent`, disk `used_percent`

18.2 Alert topic

```
# Run on: AWS CloudShell   (region = <REGION>)
TOPIC_ARN=$(aws sns create-topic --name intellia360-alerts --query TopicArn --output text)
aws sns subscribe --topic-arn "$TOPIC_ARN" --protocol email --notification-endpoint '<ALERT_EMAIL>'
echo "$TOPIC_ARN"
```

Confirm the subscription from the email AWS sends.

18.3 Alarms

```
# Run on: AWS CloudShell   (same shell as 18.2, region = <REGION>)
INSTANCE_ID='<INSTANCE_ID>'

# Hardware/host failure → automatically recover the instance, and notify
aws cloudwatch put-metric-alarm --alarm-name intellia360-api-system-check \
  --namespace AWS/EC2 --metric-name StatusCheckFailed_System \
  --dimensions Name=InstanceId,Value=$INSTANCE_ID \
  --statistic Maximum --period 60 --evaluation-periods 2 --threshold 1 \
  --comparison-operator GreaterThanOrEqualToThreshold \
  --alarm-actions "arn:aws:automate:<REGION>:ec2:recover" "$TOPIC_ARN"

# Instance (OS) check failed → notify
```

```
aws cloudwatch put-metric-alarm --alarm-name intellia360-api-instance-check \
  --namespace AWS/EC2 --metric-name StatusCheckFailed_Instance \
  --dimensions Name=InstanceId,Value=$INSTANCE_ID \
  --statistic Maximum --period 60 --evaluation-periods 3 --threshold 1 \
  --comparison-operator GreaterThanOrEqualToThreshold --alarm-actions "$TOPIC_ARN"

# CPU above 80 % for 15 minutes
aws cloudwatch put-metric-alarm --alarm-name intellia360-api-cpu \
  --namespace AWS/EC2 --metric-name CPUUtilization \
  --dimensions Name=InstanceId,Value=$INSTANCE_ID \
  --statistic Average --period 300 --evaluation-periods 3 --threshold 80 \
  --comparison-operator GreaterThanThreshold --alarm-actions "$TOPIC_ARN"

# Memory above 90 % for 10 minutes (metric from the CloudWatch agent)
aws cloudwatch put-metric-alarm --alarm-name intellia360-api-memory \
  --namespace Intellia360 --metric-name mem_used_percent \
  --dimensions Name=InstanceId,Value=$INSTANCE_ID \
  --statistic Average --period 300 --evaluation-periods 2 --threshold 90 \
  --comparison-operator GreaterThanThreshold --alarm-actions "$TOPIC_ARN"
```

Two alarms are easier to create in the console (CloudWatch → **Alarms** → **Create alarm** → **Select metric**):

- **Disk above 85 %:** namespace `Intellia360` → *InstanceId, device, fstype, path* → the row with path `/` → metric `used_percent`. Condition *Greater than 85*, action: the `intellia360-alerts` topic.
- **CloudFront 5xx:** first switch the console region to **us-east-1**, where CloudFront metrics live, and create an SNS topic there too. Then metric CloudFront → *Per-Distribution Metrics* → `5xxErrorRate` for each distribution. Condition *Greater than 5* for 5 minutes.

18.4 Uptime check

Route 53 → **Health checks** → **Create health check**:

Setting	Value
Name	`intellia360-api-health`
What to monitor	Endpoint
Protocol	HTTPS
Domain name	`<API_DOMAIN>`, port `443`
Path	`/api/health`
Create alarm	Yes → notify `<ALERT_EMAIL>`

19. Phase 15 — Backups

19.1 Choose where backups run

The repository contains `.github/workflows/database-backup.yml`. It runs the app's backup script nightly on a **GitHub-hosted runner**. **That runner cannot reach your database**, because Atlas only allows the EC2 Elastic IP (7.5). Pick one of these:

Option	How	Recommended
A. Atlas Cloud Backup	Atlas M10+: *Backup* is on by default. Set the snapshot schedule and retention under the cluster's **Backup** tab.	✅ primary
B. App backups on the server	a nightly cron on EC2 runs `npm run db:backup`, which writes an archive and uploads it to a separate S3 bucket (19.2)	✅ as a second copy
C. GitHub workflow	only works if Atlas allows GitHub's IP ranges (very broad); not recommended	❌

If you choose A and/or B, disable the GitHub workflow so it does not fail every night: **Actions** → *Database backup* → ⋯ → **Disable workflow**.

19.2 Option B: nightly backup from the server to S3

1. Create the bucket. In S3, create `intellia360-db-backups` (region `<REGION>`) with Block all public access **on**, **Versioning on**, and optionally **Object Lock** (compliance mode, e.g. 30 days). Object Lock keeps backups from being deleted or overwritten.

2. Create a write-only IAM user. The backup script signs its uploads with an access key, so it needs one (unlike the deploys). IAM → **Users** → **Create user**:

- Name `intellia360-backup-writer`, no console access.
- Attach an inline policy that allows **only uploads**:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::intellia360-db-backups/*"
    }
  ]
}
```

- Then **Security credentials** → **Create access key** → *Application running outside AWS* → copy the key id and secret.

3. Add the settings to `shared/.env`:

```
BACKUP_S3_BUCKET=intellia360-db-backups
BACKUP_S3_ACCESS_KEY_ID="<BACKUP_ACCESS_KEY_ID>"
BACKUP_S3_SECRET_ACCESS_KEY="<BACKUP_SECRET_ACCESS_KEY>"
```

```
BACKUP_S3_REGION=<REGION>
BACKUP_S3_PREFIX=intellia360
```

4. Test once, then schedule it:

```
# Run on: EC2 as intellia
cd /srv/intellia360/current && npm run db:backup
ls -lh /srv/intellia360/shared/backups/

crontab -e
```

Add these two lines (02:15 and 02:45 UTC; adjust as you like), then save:

```
15 2 * * * cd /srv/intellia360/current && npm run db:backup >> /srv/intellia360/shared/backups/backup.log
2>&1
45 2 * * * cd /srv/intellia360/current && npm run db:maintenance >>
/srv/intellia360/shared/backups/maintenance.log 2>&1
```

Restores and drills are described in `docs/DATABASE-BACKUP-RECOVERY.md`.

20. Phase 16 — Go-live checks

Run all of these. Each line shows the expected result.

20.1 From a terminal

```
# Run on: your computer

# API is up and reaches the database
curl -s https://<API_DOMAIN>/api/health; echo
#   {"status":"ok"}

# HTTP is redirected to HTTPS
curl -s -o /dev/null -w "%{http_code} %{redirect_url}\n" http://<DOMAIN>/
#   301 https://<DOMAIN>/

# Pages resolve, including a dynamic lesson id
curl -s -o /dev/null -w "%{http_code}\n" https://<DOMAIN>/login
#   200
curl -s -o /dev/null -w "%{http_code}\n" https://<DOMAIN>/student/lesson/any-lesson-id
#   200
curl -s -o /dev/null -w "%{http_code}\n" https://<DOMAIN>/this-page-does-not-exist
#   404

# Security headers are present on the frontend
curl -sI https://<DOMAIN>/login | grep -iE "content-security-policy|strict-transport-security|x-frame-
options"
#   three lines

# CORS: the frontend origin is allowed …
curl -s -o /dev/null -D - -X OPTIONS https://<API_DOMAIN>/api/auth/login \
  -H "Origin: https://<DOMAIN>" -H "Access-Control-Request-Method: POST" \
  | grep -iE "^HTTP|access-control-allow-(origin|credentials)"
#   HTTP/2 204, access-control-allow-origin: https://<DOMAIN>, access-control-allow-credentials: true

# … and any other site is refused
curl -s -w " [%{http_code}]\n" -X POST https://<API_DOMAIN>/api/auth/login \
  -H "Origin: https://evil.example" -H "Content-Type: application/json" -d '{}'
#   {"success":false,"message":"Origin not allowed"} [403]

# The server cannot be reached directly
curl -s -m 5 -o /dev/null -w "%{http_code}\n" http://<ELASTIC_IP>/ || echo "timed out (good)"
#   timed out (good)
```

20.2 In a browser (use a private window)

Step	Expected
Open `https://<DOMAIN>`	home page loads, no errors in the browser console (F12)
Open `https://<DOMAIN>/student/dashboard` while signed out	sent to `/login?next=%2Fstudent%2Fdashboard`
Sign in as the admin	lands on `/admin/dashboard`; numbers load
Open `/editor/lessons`, open a lesson	lesson editor loads its data
Register a new student account	account created, signed in, student dashboard shows
Open a lesson, then EduNotes	the PDF book opens
Ask the AI tutor a question	an answer appears (needs `GROQ_API_KEY`)
Sign out, then open `/student/dashboard`	sent back to `/login`
Reload several pages with **F5**	every page reloads (no 404s)

20.3 Final security checklist

- [ ] **No secrets in Git.** `git ls-files | grep -i '\.env'` shows only the two `.env.example` files.
- [ ] **Where secrets live.** API secrets exist only in `/srv/intellia360/shared/.env`, with mode `600` (`ls -l` shows `-rw-------`).
- [ ] **No long-lived AWS keys.** GitHub stores only `AWS_DEPLOY_ROLE_ARN`. The only access key is the backup writer's, which can upload and nothing else.
- [ ] **Server locked down.** The security group has only port 80 from the CloudFront prefix list, with no SSH.
- [ ] **S3 private.** Block Public Access is on; the bucket policy allows only your CloudFront distribution.
- [ ] **Database locked down.** Atlas network access contains only `<ELASTIC_IP>/32`, and the database user is not an Atlas admin.
- [ ] **Admin bootstrap closed.** `POST /api/admin/seed-admin` returns 410, and the demo seed was never run in production.
- [ ] **HTTPS everywhere.** Both distributions use TLSv1.2_2021 and redirect HTTP.
- [ ] **Alerts work.** The alert email subscription is confirmed.

21. Day-to-day operations

21.1 Deploying changes

Push or merge to `main`. The workflows only run when their files change:

Change touches	Workflow that runs
`apps/frontend/**`, `packages/shared/**`, `deploy/cloudfront/**`, `apps/backend/src/data/grades/*/curriculum.json`	Frontend
`apps/backend/**`, `packages/shared/**`, `deploy/deploy.sh`, `deploy/ecosystem.config.js`	Backend

Pull requests run the build and check jobs only; nothing is deployed until the merge. The API restarts during a backend deploy, so expect **a few seconds of 502 errors**.

21.2 Rolling back

Frontend: Actions → *Frontend — build & deploy…* → open an earlier successful run → **Re-run all jobs**. (With bucket versioning you can also restore individual objects in S3.)

API (the last 5 releases are kept on the server):

```
# Run on: EC2 as intellia
ls -1dt /srv/intellia360/releases/*/            # newest first
ln -sfn /srv/intellia360/releases/<PREVIOUS_RELEASE> /srv/intellia360/current
pm2 delete intellia360-api
pm2 start /srv/intellia360/current/deploy/ecosystem.config.js
pm2 save
curl -s http://127.0.0.1:4000/api/health; echo
```

The deploy script also rolls back **automatically** if a new release fails its health check.

21.3 Changing configuration

What	How
An API setting (not `NEXT_PUBLIC_*`)	edit `/srv/intellia360/shared/.env`, then `pm2 restart intellia360-api`
An API `NEXT_PUBLIC_*` setting	edit `shared/.env`, then re-run the *Backend* workflow (it rebuilds)
A frontend setting (`NEXT_PUBLIC_API_URL`, `NEXT_PUBLIC_GOOGLE_CLIENT_ID`)	change the GitHub variable, then re-run the *Frontend* workflow
Allowed origins	`CORS_ORIGIN` in `shared/.env` + restart. Also update the frontend CSP only if the **API** address changes.
Nginx config	copy the new file (11.2), then `nginx -t && systemctl reload nginx`

21.4 Database schema changes

Deploys never change the database schema. After deploying a release that changes `apps/backend/prisma/schema.prisma`:

```
# Run on: EC2 as intellia
cd /srv/intellia360/current && npm run db:indexes
```

21.5 Rotating secrets

Secret	How	Effect
`AUTH_JWT_SECRET`	new value in `shared/.env`, then restart	**everyone is signed out**
`<ORIGIN_SECRET>`	1) set the new value on the API distribution's origin header, wait until *Deployed*; 2) update the Nginx snippet and reload	brief 403s if done in the wrong order
Atlas password	new password in Atlas, update `DATABASE_URL`, restart	none
Deploy key	add a new key in GitHub, update `~/.ssh` on the server, delete the old key	none

21.6 Useful commands on the server

```
# Run on: EC2 as intellia
pm2 status                              # is the API running?
pm2 logs intellia360-api --lines 100    # recent API output
pm2 restart intellia360-api             # restart (re-reads shared/.env)
cat /srv/intellia360/current/REVISION   # which commit is live
readlink -f /srv/intellia360/current    # which release folder is live
df -h /                                 # disk space
```

```
# Run on: EC2 as root
tail -f /var/log/nginx/intellia360.error.log
systemctl status nginx
```

Ubuntu installs security updates automatically (`unattended-upgrades`). Reboot occasionally when `/var/run/reboot-required` exists; PM2 starts the API again on boot.

22. Troubleshooting

Symptom	Likely cause	Fix
Sign-in "succeeds" but you are immediately signed out, or `/api/auth/me` returns 401	frontend and API not on the same parent domain, or a `*.cloudfront.net` URL used	use `https://<DOMAIN>` and `https://<API_DOMAIN>` (2.1)
Browser console: *blocked by CORS policy*	`CORS_ORIGIN` missing this exact origin (check `www`, `https`, no trailing slash)	fix `CORS_ORIGIN`, then `pm2 restart intellia360-api`
API calls return **403** with an empty/HTML body	CloudFront's `X-Origin-Verify` value doesn't match the Nginx snippet	compare both values (11.1, 13)
API returns **403** `{"message":"Origin not allowed"}`	request came from a site not in `CORS_ORIGIN`	add the origin if it is legitimate
API returns **502**	the API process is down or restarting	`pm2 status`, `pm2 logs intellia360-api`
API returns **504**	request took longer than the origin response timeout	raise the timeout (13); check the AI provider
Endless redirects on the API	Nginx sending `X-Forwarded-Proto: http`	use the repository's Nginx file unchanged (it sends `https`)
`/login` works but `/student/lesson/<id>` shows *Not found*	router function not attached, or not published	14.2 and 14.4 → *Function associations*
Every page 403/AccessDenied	bucket policy missing or wrong distribution id	14.4 step 1
New deploy not visible	cached copy	the workflow invalidates `/*`; hard-refresh (Ctrl+F5)
EduNotes PDF doesn't open	PDF worker served with the wrong type, or the API unreachable	check `pdf.worker.min.mjs` has `Content-Type: text/javascript` in S3; check the API
Backend workflow: `AccessDeniedException` for `ssm:SendCommand`	role policy has the wrong instance id or region	16.2
Backend workflow stays *InProgress* and fails	SSM agent offline, or the deploy failed on the server	Systems Manager → Run Command → command history; CloudWatch `/intellia360/deploy`
Frontend workflow: `Not authorized to perform sts:AssumeRoleWithWebIdentity`	job not in the `production` environment, or trust policy `sub` mistyped	16.2 trust policy; environment named exactly `production`
`next build` killed on the server	out of memory	check the swap (8.1), or use `t3.medium`
Health check `{"status":"unavailable"}`	database unreachable	`DATABASE_URL`; Atlas network access has `<ELASTIC_IP>/32`

Symptom	Likely cause	Fix
Session Manager can't connect	instance role or network	7.6
Editor lesson changes disappear after a backend deploy	known issue, see section 23	apply the fix in section 23

23. Known issue to fix before relying on the editor

`apps/backend/scripts/restore-curriculum.mjs` looks up saved curriculum documents with the filter `deletedAt: null`. In Prisma's MongoDB connector that filter **skips documents where `deletedAt` was never written**, which is every normal editor save.

Every backend deploy runs this script to carry editor changes into the new release. As a result, **lesson and curriculum edits made in the editor on the server are not carried into the next release**. The edits are not lost from the database: they remain in the `curriculum_documents` collection and can be restored once the filter is fixed.

The fix is to use the same filter the application already uses elsewhere (`notDeleted()` in `apps/backend/src/lib/db-filters.ts`):

```
// apps/backend/scripts/restore-curriculum.mjs — in the bulk rehydrate query
...(INCLUDE_DELETED ? {} : { OR: [{ deletedAt: null }, { deletedAt: { isSet: false } }] }),
```

Until it is fixed, avoid editing curriculum on the live site, or commit the changed JSON files to the repository.

24. Reference

24.1 Deployment files in the repository

File	Purpose
`deploy/deploy.sh`	builds and switches an API release on EC2; health check and automatic rollback
`deploy/ecosystem.config.js`	PM2 definition: process `intellia360-api`, `127.0.0.1:4000`, working directory `apps/backend`
`deploy/nginx/intellia360.conf`	Nginx site: origin-secret check, real client IP, proxy to port 4000
`deploy/cloudwatch-agent.json`	CloudWatch agent: logs and memory/disk/swap metrics
`deploy/cloudfront/frontend-router.js`	CloudFront Function: clean URLs and dynamic-route shells
`deploy/cloudfront/frontend-security-headers.json`	CloudFront response headers policy (CSP, HSTS, …)
`.github/workflows/frontend-deploy.yml`	frontend CI/CD
`.github/workflows/backend-deploy.yml`	backend CI/CD
`.github/workflows/database-backup.yml`	scheduled backup from GitHub (see 19.1)
`apps/backend/.env.example`	API environment template
`apps/frontend/.env.example`	frontend (build-time) environment template
`docs/DEPLOYMENT-AWS.md`	condensed architecture and reference
`docs/DATABASE-BACKUP-RECOVERY.md`	backup and restore runbook

24.2 Ports and names

Thing	Value
API process (PM2)	`intellia360-api`
API listens on	`127.0.0.1:4000` (not reachable from outside)
Nginx listens on	port `80`, only from CloudFront
App user on EC2	`intellia`
App folder on EC2	`/srv/intellia360`
Health endpoint	`GET /api/health` → `{"status":"ok"}`
Session cookie	`intellia_session` (httpOnly, Secure, SameSite=Lax)
CloudFront alias hosted zone id	`Z2FDTNDATAQYW2`

24.3 GitHub configuration summary

Kind	Name	Used by
Environment	`production`	both deploy jobs
Secret	`AWS_DEPLOY_ROLE_ARN`	both deploy jobs
Variable	`AWS_REGION`	both
Variable	`EC2_INSTANCE_ID`	backend
Variable	`FRONTEND_S3_BUCKET`	frontend
Variable	`FRONTEND_CLOUDFRONT_DISTRIBUTION_ID`	frontend
Variable	`FRONTEND_ROUTER_FUNCTION_NAME`	frontend (optional)
Variable	`NEXT_PUBLIC_API_URL`	frontend build
Variable	`NEXT_PUBLIC_GOOGLE_CLIENT_ID`	frontend build (optional)
Deploy key	`intellia360-ec2` (read-only)	the EC2 server pulling code